Intermediate
Azure Lab05
Terraform Checks and Validations

Contents

Overview 1
Before You Begin 1
Phase 1 – VM Size Precondition Check 2
Phase 2 – Storage Account Name Validation 2
Phase 3 – Cleanup 4

Overview

This lab introduces Terraform validation mechanisms (preconditions and postconditions)

Before You Begin

1. Ensure you have completed Lab 0
2. Navigate to the lab directory:

   cd c:\azure-tf-int\labs\05

3. Update **main.tf** with your subscription id.
4. Update **terraform.tfvars,** adding a unique suffix to the storage account name.

Phase 1 – VM Size Precondition Check

Purpose

What we are doing:
Using Terraform's validation block inside a variable definition to restrict input values for VM sizes.

Why it matters:
In cloud environments with cost controls or policy restrictions, not all VM sizes are allowed. This validation protects the user from accidentally requesting expensive or disallowed SKUs — acting as a **client-side policy enforcement** before Terraform even contacts Azure.

Steps

1. Review the provided variables.tf file. Two variables are declared, vm_size and storage_account_name. Both have validations, the vm size must be one of the 3 listed and the storage account name must begin with “lab”
5. Review the provided terraform.tfvars file. Values for both variables are present. The vm size listed is one of the four permitted sizes and the storage account name begins with ‘lab.’
6. In terraform.tfvars; change the vm size from **"Standard_B2s_v2”**, to **"Standard_B4s”**
7. Run **terraform init** and **terraform plan**. This should **fail** with an error indicating an invalid machine size choice.
8. In **terraform.tfvars**; reset the vm size back to **Standard_B2s_v2**

Phase 2 – Storage Account Name Validation

Purpose

What we are doing:

1. **Precondition** ensures the **input value** for the storage account name starts with "lab".
2. **Postcondition** ensures that after Terraform's logic runs, the **actual deployed name** still meets this rule.

Why it matters:
Sometimes, logic inside a Terraform configuration (e.g. using replace() or lower()) transforms a valid input into an invalid output.
The precondition validates the **user's input**, while the postcondition verifies the **final evaluated result**. This double-check ensures that even advanced logic does not break compliance rules unintentionally.

Steps

1. In **terraform.tfvars;** remove '**lab**' from the storage account name
2. Run **terraform plan.** This should fail pre-condition checks with an error indicating an invalid name.
3. Revert the storage account name so it begins with 'lab' and thus complies with the validation check. Planning should now succeed as before.
4. You will now introduce an error whereby the storage account name is altered so that, whilst the initial value passes validation check, the eventual value, altered by some logic processes, is not valid.
4. In main.tf, navigate to line 75 and change the **second** instance of '**lab**' to '**test**'. This regenerates a new storage account name that varies from the name derived from the variable.
5. Run **terraform plan.** This should fail post-condition checks with an error indicating an invalid name.
6. Undo the changes to main.tf line 75
7. Run **terraform plan** followed by **terraform apply --auto-approve** to verify that the valid resources deploy successfully

Phase 3 – Cleanup

1. Run **terraform destroy**

Congratulations, you have completed this lab

©QA 2026

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026